

Chapitre 1 : Introduction

1. Méthodes classiques de vérification

Avant l'apparition des méthodes formelles, deux techniques principales étaient utilisées :

- **Les tests (jeux d'essais)** : réalisés directement sur le programme fini.
- **La simulation** : effectuée sur un **modèle théorique** avant la mise en œuvre réelle du système.

2. Limites de ces méthodes

Ces approches présentent plusieurs **inconvénients** :

1. **Coût élevé de correction** : certaines erreurs sont très coûteuses à corriger (ex. : avion, centrale).
2. **Non-exhaustivité** :
 - Les tests ne couvrent **pas tous les cas possibles**.
 - Les simulations ne garantissent **pas l'étude de toutes les situations** envisageables.

Conséquence : besoin de méthodes plus fiables et systématiques → **les méthodes formelles**.

3. Pourquoi vérifier un programme ?

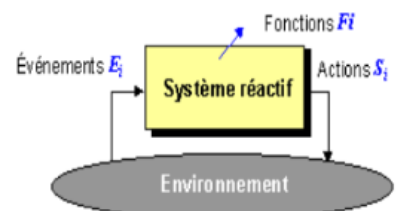
La vérification a pour but de :

1. **Garantir la fiabilité et la sûreté** des programmes : ils doivent répondre aux attentes et fonctionner sans erreurs.
2. **Éviter toute erreur** dans les **systèmes critiques**, où une défaillance peut avoir des conséquences graves.

4. Qu'est-ce qu'un système critique ?

- Ce sont des systèmes qui **contrôlent un processus** en interaction constante avec leur **environnement** (capteurs, signaux, thermomètres, etc.).
- Exemples : **aéronautique**, **téléchirurgie**, **paiement par carte à puce**, **centrale nucléaire**, etc.
- Leur rôle n'est **pas de calculer un résultat**, mais d'**assurer le fonctionnement continu** du processus → on les appelle **systèmes réactifs**.
- Ils sont souvent **temps réel** et **distribués**.

Un **système réactif** est un système ouvert répondant constamment aux sollicitations de son environnement en produisant des actions sur celui-ci.



Les réactions S_i aux événements E_i sont générées par des fonctions F_i du système qui peuvent chacune être soumises à des contraintes temporelles d_i imposées par l'environnement.

Exemples de systèmes critiques

Les **systèmes critiques** sont ceux dont une défaillance peut avoir des **conséquences graves** :

- **Pacemaker** (stimulateur cardiaque)
- **Pilote automatique d'un avion**
Une mauvaise vérification peut entraîner des **dommages matériels, financiers, ou humains**.

Exemples de catastrophes dues à une mauvaise vérification

1. Ariane V – Vol 501 (4 juin 1996)

- 40 secondes après le lancement, la fusée s'**auto-détruit**.
Causes principales :
 1. Réutilisation d'un code d'Ariane IV sans adaptation.
 2. Conversion d'un **flottant 64 bits** en **entier signé 16 bits**.
 3. Valeur du flottant trop grande pour être convertie.
 4. **Erreur dans le système de positionnement**.
 5. Le système a « corrigé » à tort la trajectoire.
 6. Résultat : la fusée s'est **brisée en deux**.

Conséquence : plusieurs centaines de millions d'euros perdus à cause d'une erreur logicielle.

2. Therac-25 (accélérateur médical)

- Appareil utilisé pour traiter des tumeurs par rayons.
Problèmes :
 - La **sécurité reposait uniquement sur le logiciel**.
 - Plusieurs **erreurs de programmation** ont entraîné des **surdoses massives** de radiation.
- Bilan :**
 - 6 accidents avérés
 - 4 morts
 - 2 ans pour identifier le logiciel comme responsable=>Machines retirées du marché.

Vérification vs Validation formelle

Concept	Question posée	Objectif	Outils
Vérification formelle	« Est-ce que le programme est correct , sans bug ? »	Vérifier la cohérence interne du programme ou du modèle.	Prouveurs de théorèmes (ex. : Coq)

Validation formelle	« Est-ce que le programme répond bien aux exigences des utilisateurs ? »	Vérifier la conformité aux spécifications fonctionnelles.	Animation / Model checking
----------------------------	---	---	-----------------------------------

Différence clé :

- La **vérification** garantit la **justesse interne**.
- La **validation** garantit la **conformité externe**.

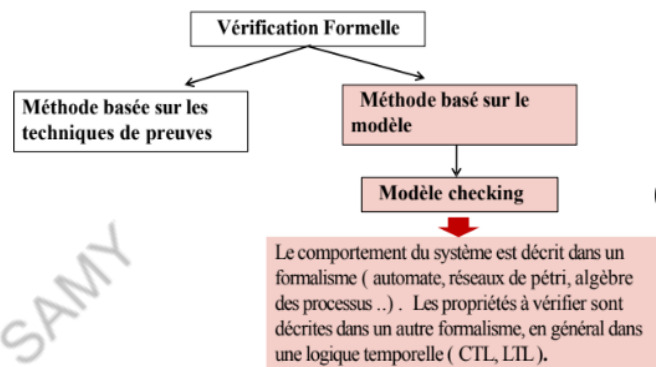
Vérification formelle – Méthodes principales

1. Méthodes basées sur la preuve

- Utilisent des **techniques de raisonnement mathématique** (preuves de théorèmes).
- Exemples : Coq, Isabelle, HOL, etc.

2. Méthodes basées sur le modèle (Model Checking)

- Le **comportement du système** est représenté sous forme d'un **modèle formel** :
 - Automates
 - Réseaux de Pétri
 - Algèbres de processus
- Les **propriétés à vérifier** sont exprimées dans une **logique temporelle** :
 - CTL (Computation Tree Logic)
 - LTL (Linear Temporal Logic)



Principe du Model Checking

1. **Construire un modèle M** décrivant le **comportement du système** (ex. : système à états finis).
2. **Exprimer les propriétés Φ** à vérifier dans une **logique temporelle**.
3. **Vérifier si :**
 $M \models \Phi$
 (c'est-à-dire : « le modèle M satisfait la propriété Φ »).

Si la propriété est fausse, le model checker fournit souvent un **contre-exemple** (une séquence d'états montrant l'erreur).

Étapes de la vérification par Model Checking

1. **Définition des propriétés à vérifier**
→ Identifier ce que le système doit respecter (ex. : sécurité, absence de blocage, bonne séquence d'actions...).
2. **Construction du modèle formel**
→ Représenter le comportement du système sous forme d'un **modèle mathématique** (automate, réseau de Pétri, etc.).
3. **Vérification automatique**
→ Exécuter l'algorithme de **model checking** pour voir si le modèle **satisfait les propriétés définies**.
 - Si c'est le cas : le système est conforme.
 - Sinon : l'outil fournit un **contre-exemple** (une séquence d'états illustrant la violation).

MAGUEMOUN SAMY

Chapitre 2 : Systèmes de transitions

1/Comportement d'un système

- Les **systèmes de transitions** sont les **modèles principaux** utilisés pour représenter le **comportement des systèmes réactifs** et/ou **temps réel**.
- Un **système de transition** est en réalité un **automate**, c'est-à-dire un modèle mathématique décrivant **l'évolution d'un système dans le temps**.
- Les **automates** sont employés dans :
 - la **théorie des langages**,
 - la **modélisation** et la **vérification** de systèmes communicants (souvent temps réel).

Description du comportement

Elle consiste à spécifier **comment le système évolue dans le temps** en identifiant :

- **Les actions** : opérations que le système exécute.
Exemples : usinage d'une pièce, ouverture de fichier, écriture en base de données.
- **Les événements** : stimuli déclencheurs d'actions.
Exemples : dépassement de température, réception d'un message, ouverture de porte.
- **Les contraintes temporelles** : délais à respecter.
Exemples : trouver un vol en moins de 2 min, envoyer un message toutes les 100 s.
- **Les conditions d'activation** : règles logiques nécessaires avant d'agir.
Exemples : si le nombre de pièces = 10 → fermer la boîte.
- **Les situations anormales (exceptions)** : erreurs et traitements associés.
Exemples : ascenseur bloqué, appui de touche sans effet.
- **Les états significatifs du système** : différentes phases de fonctionnement.
Exemples : ascenseur à l'arrêt, en mouvement, en maintenance.

Remarque

La **modélisation du comportement** est **complexe**, car :

1. Le nombre de situations possibles peut être **infini**.
2. Les **concepts à considérer** sont nombreux (actions, événements, temps, états...).
3. Il **n'existe pas de modèle universel** valable pour tous les types de systèmes.

2/Automates à états finis (AEF)

Définition

Un **automate à états finis** est défini par un **quintuplet** : $A=(S,E,T,S_0,F)$ où :

- **S** : ensemble fini d'**états**
- **E** : **alphabet** (ensemble des symboles ou événements)

- **T : fonction de transition** $T: S^*E \rightarrow S$
→ décrit les passages d'un état à un autre selon un événement
- **S₀ : état initial** (point de départ)
- **F : ensemble des états finaux** (ou d'acceptation)

♦ Représentation graphique

Un **automate** se représente par un **graphe orienté** :

- **Les nœuds** → les **états** du système.
- **Les arcs fléchés** → les **transitions**, étiquetées par les événements de l'alphabet.

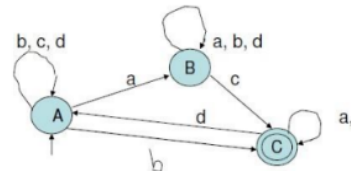
$$S = \{ A, B, C \}$$

$$E = \{ a, b, c, d \}$$

$$T = \{ (A, a, B), (A, b, C), (A, b, A), (A, c, A), (A, d, A), (B, c, C), (B, a, B), (B, b, B), (B, d, B), (C, d, A), (C, a, C), (C, b, C), (C, c, C) \}$$

$$S_0 = \{ A \}$$

$$F = \{ C \}$$



3/Système de Transition

♦ Définition 1 : Système de Transition Étiqueté (STE)

Un **STE** est défini par le **quadruplet** $A = (E, Q, q_0, T)$ où :

- **E** : ensemble fini d'**étiquettes de transition** (les actions ou événements déclencheurs),
- **Q** : ensemble fini d'**états**,
- **$T \subseteq Q \times E \times Q$: relation de transition**, reliant les états via les étiquettes,
- **$q_0 \in Q$: état initial.**

Remarque

Un **système de transition étiqueté** est similaire à un **automate à états finis**, mais **sans états finaux**.

Il modélise uniquement le **comportement** du système, pas la reconnaissance d'un langage.

♦ Définition 2 : Système de Transition (ST)

Un **ST** est défini par le **triplet** $A = (Q, q_0, T)$ où :

- **Q** : ensemble fini d'**états**,
- **$T \subseteq Q \times Q$: relation de transition** (non étiquetée),
- **$q_0 \in Q$: état initial.**

Remarque

- Dans un système de transitions (étiqueté ou non), il peut exister **plusieurs états initiaux**, notés $I \subseteq Q$.

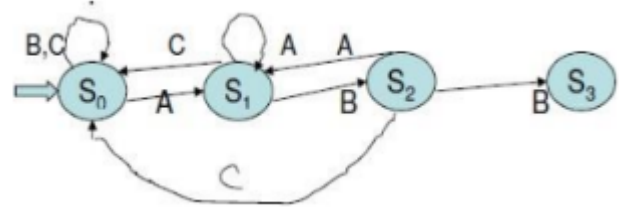
Exemple : le digicode

On considère un **digicode** permettant d'ouvrir une porte.

- Les touches possibles : **A, B, C**.
- Le code correct est la **séquence A → B → B**.
- La porte s'ouvre dès que cette suite est saisie, **même si des erreurs ont été faites avant**.

Ce système peut être modélisé comme un **système de transition étiqueté**, où :

- Les **états** représentent la progression dans la saisie du code,
- Les **étiquettes** représentent les **touches appuyées**,



4/États et Transitions

4.1/Les États

- Un **état** représente une **situation particulière** du système à un instant donné.
 - ♦ Exemples :
 - « Connexion établie » (dans un protocole réseau)
 - « Usinage de pièce en cours » (dans un système industriel)
- ♦ **Caractéristiques :**
 - C'est une **abstraction** : elle permet de **simplifier** et de **masquer les détails internes** du système.
 - Le niveau de détail dépend du **besoin de modélisation** :
 - Exemple : « mémoire = 0 » (simple) vs « duplication de base terminée » (complexe).
 - Les **noms d'états** peuvent être des **numéros** ou des **chaînes de caractères**, selon la **lisibilité souhaitée**.

En résumé : un état = une situation du système, représentée de façon abstraite et lisible.

4.2/Les Transitions

Une **transition** représente le **passage d'un état à un autre** en fonction de certains événements ou conditions.

Chaque transition est **étiquetée**, et son étiquette peut comporter plusieurs éléments :

- ♦ **Composition d'une étiquette :**
 1. **Événements** → déclenchent la transition (ex. : réception d'un signal).
 2. **Garde (condition)** → doit être vraie pour autoriser le passage (ex. : $x > 5$).
 3. **Actions** → opérations effectuées lors du changement d'état (ex. : remise à zéro d'une horloge, mise à jour d'une variable).

Forme générale d'une étiquette :

`<etiquette>:=[<garde>][<liste d'événements>]`

[<condition sur l'horloge>;][<liste d'actions>;]

♦ **Remarque**

La **forme exacte** de l'étiquette dépend du **type d'automate** :

- Automate simple
- Automate avec variables
- Automate temporisé
- Automate avec échanges de messages
(détails vus en travaux dirigés)

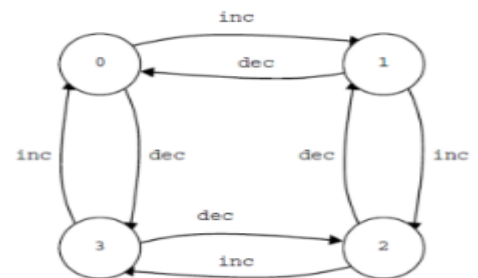
4.3/Exemples d'automates simples

Exemple 1 – Montre numérique (hh:mm)

- $24 \times 60 = 1440$ **états** possibles (un par minute de la journée).
- Les transitions ne sont **pas étiquetées** : le passage d'un état à un autre se fait **chaque minute automatiquement**.

Exemple 2 – Compteur modulo 4

- États possibles : **0, 1, 2, 3**.
- Transitions :
 - **inc** → incrémentation ($0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$)
 - **dec** → décrémentation ($3 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 3$).



♦ **Automate déterministe et indéterministe**

- **Automate déterministe :**

Un automate est **déterministe** si, pour chaque couple (état, étiquette), **une seule transition** est possible.

Cela signifie qu'à partir d'un même état et d'une même action, il n'existe qu'un seul chemin possible.

- **Automate indéterministe :**

Il est **indéterministe** lorsqu'il existe **au moins un état** avec **plusieurs transitions ayant la même étiquette**, mais menant à **des états différents**.

Donc, plusieurs chemins sont possibles pour une même action.

- **Exemple :**

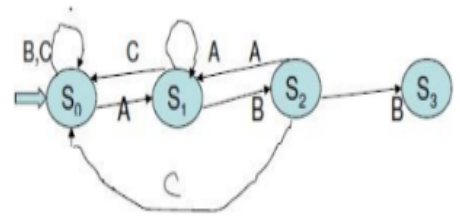
Le digicode qui ouvre une porte une seule fois peut être modélisé comme un automate. Selon la suite des touches, on peut avoir plusieurs transitions possibles pour une même entrée.

♦ **Chemin d'exécution (ou Trace)**

- Un **chemin d'exécution** (ou **trace**) représente une **séquence de transitions** parcourues par le système lors de son fonctionnement.
- Autrement dit, c'est **la suite des actions (ou étiquettes)** exécutées.

- **Exemple :**

Les séquences **BAABABB**, **AABABB**, **CBACAABB** sont des **traces d'exécution** possibles du modèle de digicode permettant d'ouvrir la porte



- ◆ **Chemins d'exécution**

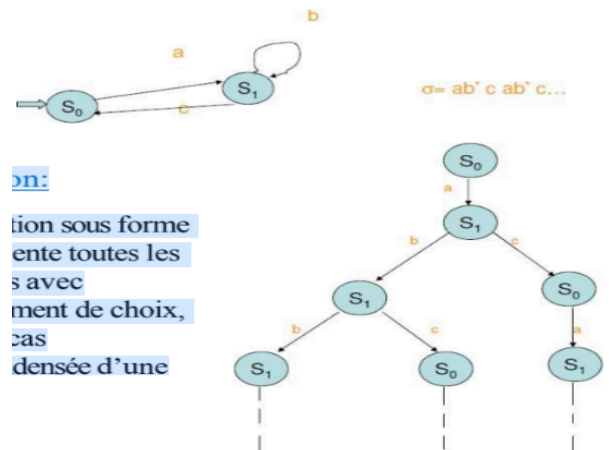
- Un **chemin** peut être vu comme :
 - une **séquence d'états** ($S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3$), ou une **séquence de transitions**.
- Exemple : la séquence **S0; S1; S2; S3** représente un chemin qui **ouvre la porte**.
- **Remarque :**
 - Les **exécutions qui ouvrent la porte** sont **finies** (le système atteint un but).
 - Les **exécutions qui ne l'ouvrent pas** sont **infinies** (le système tourne sans fin).

- ◆ **Définitions importantes**

- **Exécution partielle :**
Chemin qui **commence à l'état initial** de l'automate, mais **peut s'arrêter avant la fin**.
- **Exécution complète :**
Chemin **qui ne peut plus être prolongé**, soit parce que :
 - le système est **bloqué**, ou l'exécution est **infinie**.

- ◆ **Arbre d'exécution**

- L'**arbre d'exécution** représente l'**ensemble de toutes les exécutions possibles** du système. Il décrit **toute la dynamique** du système.
- Cet arbre peut être **infini**, car il inclut toutes les combinaisons de transitions possibles.
- Sous forme d'arbre, on visualise **tous les choix possibles à chaque étape**, ce qui est plus précis qu'une simple trace écrite.



- ◆ **Séquence**

- Soit **S** un ensemble d'états :
 - $S^* \rightarrow$ ensemble des **séquences finies** sur S.
 - $S^\omega \rightarrow$ ensemble des **séquences infinies** sur S.
 - $\sigma_i \rightarrow$ le **i^e élément** (à partir de 0) d'une séquence σ .
- Une **séquence σ** est notée **<s1, s2, ...>**.
 - La **séquence vide** est notée **<>**.

♦ Notations importantes

- σ^* : ensemble de **toutes les répétitions possibles** (0 ou plus) de la séquence σ .
- σ^+ : ensemble de **répétitions non vides** de $\sigma \rightarrow (\sigma^*$ sans la séquence vide).
- $\sigma\omega$: **répétition infinie** de la séquence σ .

♦ Traces finies

- Soit un système de transitions (S, I, R) :
 - S : ensemble des états,
 - $I \subseteq S$: ensemble des états initiaux,
 - R : relation de transitions.
- Une **trace finie** est une **séquence finie d'états** $\sigma \in S^*$
qui **représente un chemin valide** du système (chaque état mène au suivant via R).

$$\sigma = \langle S_0 \rightarrow S_1 \rightarrow \dots \rightarrow S_{n-1} \rightarrow S_n \rangle$$

$$\forall i \in [0..n[: (S_i, S_{i+1}) \in R$$

♦ Trace finie maximale

- Une **trace finie** est dite **maximale** si elle **ne peut plus être prolongée**.
- Autrement dit, le **dernier état** de la trace (S_n) **n'a pas de successeur** dans la relation de transitions R . Elle va **aussi loin que possible** dans le système.

♦ Trace infinie

- Une **trace infinie** est une **séquence infinie d'états** (dans $S\omega$)
où **chaque état mène au suivant** selon la relation de transitions R .
- Elle représente une **exécution sans fin** du système (ex. : un processus en boucle).

$$tr = \langle S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \dots \rangle$$

$$\forall i \in \mathbb{N} : (S_i, S_{i+1}) \in R$$

♦ Conclusion

- Pour un système de transitions (S, I, R) et un état $s \in S$:
Traces(s) = ensemble de **toutes les traces finies maximales** et **traces infinies** commençant à s .

♦ Exécution

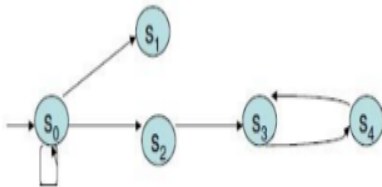
- Une **exécution** d'un système de transitions (S, I, R) est :
 - soit une **trace infinie**,
 - soit une **trace finie maximale**,
 - dont le premier état s_0 appartient à l'ensemble des états initiaux I .**
- L'ensemble de toutes les exécutions du système est noté : **Exec(S)**.

- Il existe une **seule exécution vide (ϵ)** si et seulement si l'ensemble des états initiaux **I** est vide ($I = \emptyset$).

♦ Préfixe d'exécution

- Un **préfixe d'exécution** est une **trace finie** issue d'une exécution, commençant par un **état initial** $s_0 \in I$, et se terminant par un **état final** S_n du préfixe.

Exemple 1 : Traces



$$\text{Traces}(s_1) = \{ \langle s_1 \rangle \}$$

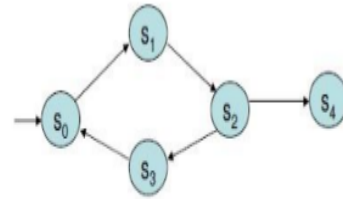
$$\text{Traces}(s_3) = \{ \langle (s_3 \rightarrow s_4)^\omega \rangle \}$$

$$\text{Traces}(s_2) = \{ \langle s_2 \rightarrow (s_3 \rightarrow s_4)^\omega \rangle \}$$

$$\text{Traces}(s_0) = \{ \langle s_0^\omega \rangle, \langle s_0^+ \rightarrow s_1 \rangle, \langle s_0^+ \rightarrow s_2 \rightarrow (s_3 \rightarrow s_4)^\omega \rangle \}$$

$$\text{Exec}(S) = \text{Traces}(s_0).$$

Exemple 2 : Traces, Exécutions



$$\text{Traces}(s_2) = \{ \langle s_2 \rightarrow (s_3 \rightarrow s_0 \rightarrow s_1 \rightarrow s_2)^* \rightarrow s_4 \rangle, \langle (s_2 \rightarrow s_3 \rightarrow s_0 \rightarrow s_1)^\omega \rangle \}$$

$$\text{Traces}(s_0) = \{ \langle s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow (s_3 \rightarrow s_0 \rightarrow s_1 \rightarrow s_2)^* \rightarrow s_4 \rangle, \langle (s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow s_3)^\omega \rangle \}$$

$$\text{Exec}(S) = \text{Traces}(s_0).$$

Exemple 3 : traces, exécutions



$$\text{Traces}(s_2) = \{ \langle s_2 \rightarrow (s_3 \rightarrow s_2)^* \rightarrow s_4 \rangle, \langle (s_2 \rightarrow s_3)^\omega \rangle \}$$

$$\text{Traces}(s_0) = \{ \langle s_0 \rightarrow s_2 \rightarrow (s_3 \rightarrow s_2)^* \rightarrow s_4 \rangle, \langle s_0 \rightarrow (s_2 \rightarrow s_3)^\omega \rangle \}$$

$$\text{Traces}(s_1) = \{ \langle s_1 \rightarrow s_3 \rightarrow s_2 \rightarrow (s_3 \rightarrow s_2)^* \rightarrow s_4 \rangle, \langle s_1 \rightarrow (s_3 \rightarrow s_2)^\omega \rangle \}$$

$$\text{Exec}(S) = \text{Traces}(s_0) \cup \text{Traces}(s_1).$$

♦ Atteignabilité (ou accessibilité)

- Un **état est atteignable** s'il apparaît **au moins une fois** dans l'**arbre d'exécution** de l'automate.
- Cette notion est essentielle pour **vérifier des propriétés** du système.

- Si un état particulier est atteignable, cela signifie que la **propriété associée** peut être réalisée.

Exemple :

Si un automate possède un état « **Porte_Fermée** », alors prouver que « la porte peut être fermée » revient à montrer que **cet état est atteignable**.

Formellement :

Dans un système de transitions **(S, I, R)** :

- Un état $s \in S$ est **accessible** s'il existe un **préfixe d'exécution** dont s est l'état final.
- **Acc(S)** = ensemble de tous les **états accessibles** du système.

♦ État récurrent

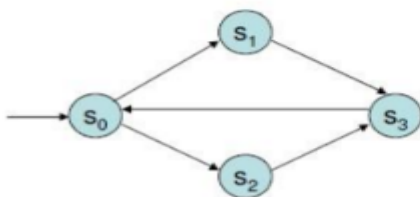
- Un **état récurrent** est un état qui **réapparaît régulièrement** au cours d'une exécution.

Cas :

1. **Exécution infinie (σ infinie)** :
→ Un état s est **récurrent** s'il apparaît **une infinité de fois** dans la séquence d'états.
2. **Exécution finie (σ finie)** :
→ Un état s est **récurrent** si, à partir d'un certain indice i , il reste **constamment présent** (il devient l'état final).

Notation :

- **Inf(σ)** = ensemble des **états récurrents** dans une exécution σ .



s_1 est récurrent dans $\langle (s_0 \rightarrow s_1 \rightarrow s_3)^\omega \rangle$

s_1 est récurrent dans $\langle (s_0 \rightarrow s_1 \rightarrow s_3 \rightarrow s_0 \rightarrow s_2 \rightarrow s_3)^\omega \rangle$

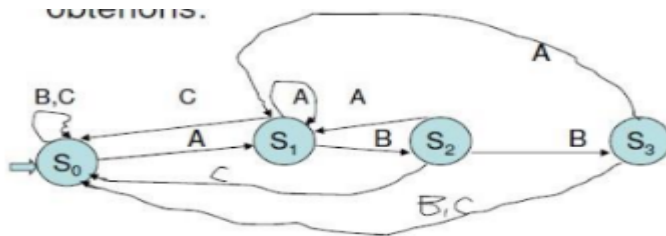
s_1 n'est pas récurrent dans $\langle (s_0 \rightarrow s_1 \rightarrow s_3)^* \rightarrow (s_0 \rightarrow s_2 \rightarrow s_3)^\omega \rangle$

♦ Exemple : Digicode avec code ABB

Si l'on modélise un **digicode** qui ouvre une porte lorsque la séquence **ABB** est saisie, et que l'on suppose que ce code peut être entré **un nombre infini de fois**,

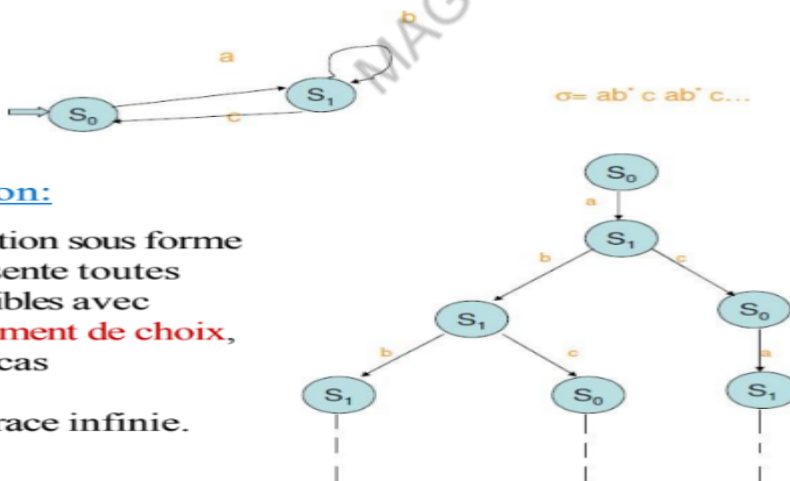
alors le **système de transitions** obtenu représentera une **exécution infinie**, où la séquence **ABB** se répète sans fin (**ABBABBABB...**).

Cela illustre qu'un **état récurrent** est atteint à chaque fois que la porte s'ouvre à nouveau.



♦ Conclusion — Comportement d'un système

- Le **comportement d'un système** peut être représenté par un **chemin** dans un **automate A**, c'est-à-dire une **suite ordonnée de transitions** $(q_i, e_i, q'_i)(q_{i+1}, e_{i+1}, q'_{i+1})$ qui s'enchaînent correctement.
- La **longueur** du chemin $|\sigma|$ correspond au **nombre de transitions**, et elle peut être **finie ou infinie**.
- Pour **analyser ou suivre** le comportement du système, il est utile de le **tracer** sous forme **d'arbre d'exécution**.
- L'**arbre d'exécution** permet de **visualiser toutes les séquences possibles** et les **choix effectués à chaque étape**, contrairement à une simple trace condensée (linéaire).

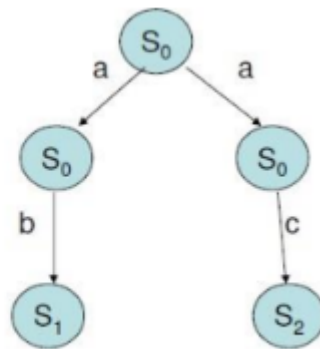
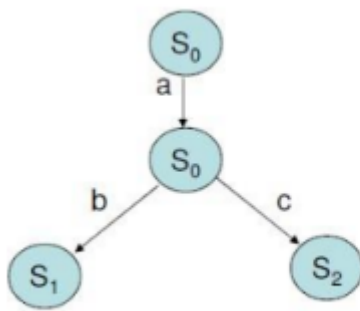


ion:

ation sous forme
ésente toutes
sibles avec
oment de choix,
cas

trace infinie.

Un autre exemple :



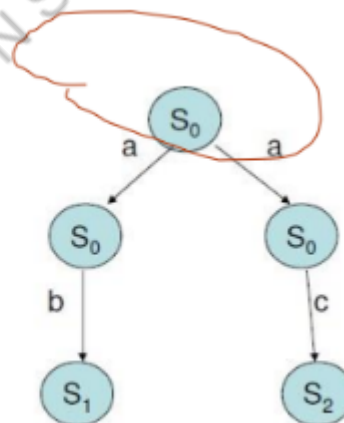
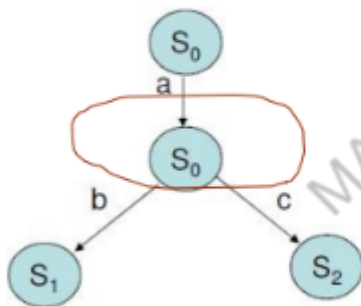
♦ Équivalence des arbres d'exécution

Deux **arbres d'exécution** peuvent **générer les mêmes mots (ou chaînes)**, c'est-à-dire qu'ils produisent **les mêmes séquences d'actions ou d'états**.

Cependant, ils **ne sont pas équivalents en termes de branchement**, car **le moment où les choix sont effectués** dans chaque arbre **n'est pas le même**.

Un autre exemple :

autre Exemple:



♦ Équivalence d'un STE à un ST sans étiquette

- Deux **arbres d'exécution** peuvent produire **les mêmes mots (ou séquences d'états)** : ils sont donc **équivalents du point de vue des choix possibles**.
- Cependant, ils **ne sont pas équivalents en termes de branchement**, car **le moment où les choix sont faits** diffère d'un arbre à l'autre.
- De la même façon, un **Système de Transition Étiqueté (STE)** peut être **représenté par un Système de Transition (ST) sans étiquette**, **sans perte de comportement observable** : les transitions restent identiques, mais on ignore simplement les étiquettes.

- $S' = (\{\epsilon\} \cup E) \times S$
- $I' = \{\epsilon\} \times I$
- $R' = \{ ((e, s), (e', s')) : (s, s') \in R \text{ \& } e, e' \in E \text{ tq } e \text{ \& } e' \text{ \> étiquette de l'arc } (s, s') \}$

The figure contains two state transition diagrams. The left diagram has four states: s_0 , s_1 , s_2 , and s_3 . Transitions are: $s_0 \xrightarrow{a} s_1$, $s_0 \xrightarrow{b} s_2$, $s_1 \xrightarrow{c} s_3$, $s_2 \xrightarrow{d} s_3$, and a direct transition $s_0 \xrightarrow{c} s_3$. The right diagram has five states: (ϵ, s_0) , (a, s_1) , (b, s_2) , (c, s_3) , and (d, s_3) . Transitions are: $(\epsilon, s_0) \xrightarrow{a} (a, s_1)$, $(\epsilon, s_0) \xrightarrow{b} (b, s_2)$, $(a, s_1) \xrightarrow{c} (c, s_3)$, $(b, s_2) \xrightarrow{d} (d, s_3)$, and a direct transition $(\epsilon, s_0) \xrightarrow{c} (c, s_3)$.

- Une **structure de Kripke** est une **extension d'un système de transitions**, à laquelle on ajoute une **fonction d'étiquetage** qui associe à chaque **état** les **propriétés vraies** dans cet état.
- Elle est définie par le **quintuplet** :

$$K = (E, Q, T, q_0, I)$$
 où :
 - **E** : ensemble fini d'étiquettes de transition,
 - **Q** : ensemble fini d'états,
 - $T \subseteq Q \times E \times Q$: relation de transitions,
 - $q_0 \in Q$: état initial,
 - **I** : fonction qui associe à chaque état **s** l'ensemble des **propriétés vraies** dans **s**.
- Les **structures de Kripke** sont utilisées pour **modéliser les systèmes** dans les méthodes formelles, notamment en **logique temporelle** et **model checking**, à la place des systèmes de transitions classiques.

Lorsqu'on décrit un système complexe, on le décompose souvent en **plusieurs sous-systèmes** (appelés systèmes de transitions ou structures de Kripke), puis on les combine à l'aide de **produits** pour obtenir le système global.

♦ 1. Produit libre (ou produit cartésien)

- On considère plusieurs systèmes de transitions :
 $A_i = (Q_i, E_i, T_i, q_{0i}, l_i)$ pour $i=1, \dots, n$.
- On introduit une **étiquette fictive "-"** pour indiquer qu'un sous-système **ne fait pas de transition** pendant qu'un autre en fait (transition de bégaiement).
- Le **produit libre** $A_1 \times A_2 \times \dots \times A_n$ est un nouveau système :
 - États** : combinaison des états de chaque sous-système $Q = Q_1 \times Q_2 \times \dots \times Q_n$
 - Étiquettes** : ensemble de toutes les étiquettes $E = |$
 - État initial** : $q_0 = (q_{01}, q_{02}, \dots, q_{0n})$
 - Transitions** : chaque transition combine celles des sous-systèmes
 - Interprétation** : chaque sous-système évolue **indépendamment** des autres.

♦ 2. Produit synchronisé

- Pour synchroniser les sous-systèmes, on définit un **ensemble de synchronisation (Sync)** qui regroupe les actions à effectuer **simultanément**.
- Seules les transitions dont les étiquettes appartiennent à **Sync** sont conservées.
 On obtient un **produit synchronisé** noté :
 $(A_1 \times A_2 \times \dots \times A_n)$

♦ Produits synchrone et asynchrone

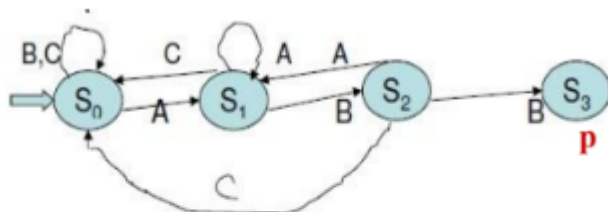
- Produit synchrone** :
 Les automates évoluent **en même temps** (exemple : feux de signalisation).
- Produit asynchrone** :
 Les automates évoluent **indépendamment** les uns des autres.

♦ 3. Propriétés à vérifier (structures de Kripke)

- On associe à chaque état des **propriétés élémentaires vraies ou fausses**, selon le comportement du système.
- Exemple :

Pour un **digicode** :

- p = "la porte est ouverte"
- p est **vrai** à l'état **S3**
- p est **faux** à **S0, S1, S2**



p	S0	S1	S2	S3
Valeur de vérité	0	0	0	1

♦ Propriétés complexes à vérifier

Pour vérifier le comportement d'un système, on part des **propriétés élémentaires** et on construit des **propriétés plus complexes**, souvent exprimées en **logique temporelle**.

Les principales familles de propriétés sont :

1 Propriété d'atteignabilité

- Énonce qu'une **situation particulière peut être atteinte** dans le système.
- **Exemples :**
 - « On ne peut pas atteindre l'état crash »
 - « On peut entrer en section critique »
 - « On peut toujours revenir à l'état initial »

2 Propriété de sûreté

- Garantit que **quelque chose de mauvais ne se produira jamais** sous certaines conditions.
- **Exemples :**
 - « Les deux systèmes ne sont jamais en section critique »
 - « Il n'y aura jamais de débordement mémoire »
- **Remarque :**
 - La négation d'une propriété d'atteignabilité simple est une propriété de sûreté, et vice versa.

3 Propriété de vivacité

- Assure que **quelque chose finira par se produire** sous certaines conditions.
- **Exemples :**
 - « Toute requête finira par être satisfaite »
 - « La cabine d'un ascenseur arrivera un jour »
 - « Le feu passera au vert »
 - « Le programme se terminera »

4 Propriété d'équité

- Variante de la vivacité : assure qu'un événement **aura lieu de manière répétée** (ou non).
- **Exemples :**
 - « La barrière sera levée infiniment souvent »
 - « L'accès à la section critique sera accordé un nombre infini de fois si demandé infiniment »
- Parfois appelée **vivacité répétée** ou **atteignabilité répétée**.

5] Propriété d'absence de blocage

- Garantit que le système **ne se retrouve jamais dans une situation où il ne peut plus progresser.**
- Très importante pour **les systèmes continus ou non terminables** (ex. : réseaux de capteurs pour incendie).
- Souvent considérée comme une **propriété de sûreté**, car elle stipule qu'**une situation indésirable ne se produira jamais.**

MAGUEMOUN SAMY

Chapitre 3 : Logique Temporelle

1. Introduction

La logique temporelle étend la logique classique pour raisonner sur des propriétés dépendant de l'évolution dans le temps. Elle permet d'exprimer des notions comme « toujours », « éventuellement », « jusqu'à ce que », sans représenter explicitement le temps.

Les propriétés sont vérifiées sur un modèle (automate, structure de Kripke, etc.).

Deux types principaux :

- **LTL (Linear Temporal Logic)** : raisonnement le long d'une exécution unique (vue comme une ligne temporelle).
- **CTL (Computation Tree Logic)** : raisonnement sur un ensemble d'exécutions formant un arbre.

2. Langage et éléments

- **Propositions atomiques** : décrivent les propriétés vraies dans un état.
Exemple : si $x = 2$ dans l'état q_1 , alors la proposition $(x = 2)$ est vraie dans q_1 .
- **Connecteurs logiques classiques** : $\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \forall, \exists$
- **Opérateurs temporels** :
 - $X P$: P est vrai à l'état suivant.
 - $F P$: P sera vrai à un moment futur.
 - $G P$: P est toujours vrai à partir de maintenant.
 - $P1 \text{ U } P2$: $P1$ reste vrai jusqu'à ce que $P2$ devienne vrai.

Compositions fréquentes :

- $GF P$: P est vrai infiniment souvent.
- $FG P$: à partir d'un moment, P sera toujours vrai.
- **$P1 \text{ U } P2$ (Until)** : $P1$ reste vrai jusqu'à ce que $P2$ devienne vrai.

Relations utiles :

- $G P \equiv \neg F \neg P$
- $F P \equiv \text{True} \text{ U } P$

3. Quantification sur les exécutions (CTL)

- **$A P$** : P est vrai sur **toutes les exécutions** à partir de l'état courant.
- **$E P$** : il existe au moins **une exécution** où P est vrai.
- **Équivalences** :
 - $A P \equiv \neg E \neg P$
 - $E P \equiv \neg A \neg P$

Distinction essentielle :

- **$A P$** : sur **tous les chemins possibles**

- $G P$: sur **tous les instants d'une exécution particulière**

4. Combinaisons entre A/E et F/G + opérateurs CTL

Opérateur	Signification	Interprétation
$EF P$	Il existe au moins une exécution dans laquelle P deviendra vraie	P est éventuellement possible
$AF P = \neg E \neg F P$	Dans toutes les exécutions, P deviendra vraie	P arrivera forcément
$EG P = E \neg F \neg P$	Il existe une exécution dans laquelle P est toujours vraie	P peut rester vrai indéfiniment sur un chemin
$AG P = \neg EF \neg P$	Dans toutes les exécutions, P est toujours vraie	P est toujours vrai, aucun chemin ne contredit
$EX P$	Au prochain état, au moins un vérifie P	Il existe un état suivant où P est vrai
$AX P$	Tous les états successeurs vérifient P	P vrai dans tous les états suivants
$E[P U Q]$	Il existe une exécution où P reste vrai jusqu'à Q	Jusqu'à l'arrivée de Q sur un chemin existant
$A[P U Q]$	Dans toutes les exécutions, P reste vrai jusqu'à Q	P reste vrai jusqu'à ce que Q devienne vrai partout

Transformations importantes :

- $EF P = E (\text{True} \cup P)$
- $AF P = A (\text{True} \cup P)$
- $EG P = \neg(A (\text{True} \cup \neg P))$
- $AG P = \neg(E (\text{True} \cup \neg P))$

5. Syntaxe de la logique temporelle

- Formule d'état élémentaire : p
- Fermeture par connecteurs classiques : $\neg\Phi, \Phi \wedge \Psi, \Phi \vee \Psi, \Phi \Rightarrow \Psi, \Phi \Leftrightarrow \Psi$
- Opérateurs temporels : $X\Phi, F\Phi, G\Phi, \Phi U \Psi$
- Quantificateurs de chemins (CTL) : $A\Phi, E\Phi$

Grammaire formelle :

$\Phi ::= p_1 \mid p_2 \mid \dots$ (propositions atomiques)

$\Phi ::= \neg\Phi \mid \Phi \vee \Psi \mid \Phi \wedge \Psi \mid \Phi \Rightarrow \Psi \mid \Phi \Leftrightarrow \Psi$ (combinateurs booléens)

$\Phi ::= X\Phi \mid F\Phi \mid G\Phi \mid \Phi U \Psi$ (combinateurs temporels)

$\Phi ::= A\Phi \mid E\Phi$ (quantificateurs de chemins)

6. Sémantique et relation de satisfaction

Pour une trace σ et un instant i :

1. Proposition atomique

$$\sigma, i \models p \iff p \in l(\sigma[i])$$

2. Négation

$$\sigma, i \models \neg p \iff \text{il n'est pas vrai que } \sigma, i \models p$$

3. Conjonction

$$\sigma, i \models p \wedge q \iff \sigma, i \models p \text{ et } \sigma, i \models q$$

4. Disjonction

$$\sigma, i \models p \vee q \iff \sigma, i \models p \text{ ou } \sigma, i \models q$$

5. Implication

$$\sigma, i \models p \Rightarrow q \iff \text{si } \sigma, i \models p \text{ alors } \sigma, i \models q$$

6. Next (X)

$$\sigma, i \models Xp \iff i < |\sigma| \text{ et } \sigma, i+1 \models p$$

7. Finally / Eventually (F)

$$\sigma, i \models Fp \iff \exists j, i \leq j \leq |\sigma| \text{ tel que } \sigma, j \models p$$

8. Globally / Always (G)

$$\sigma, i \models Gp \iff \forall k, i \leq k \leq |\sigma|, \sigma, k \models p$$

9. Until (U)

$$\sigma, i \models pUq \iff \exists j, i \leq j < |\sigma|, \sigma, j \models q \text{ et } \forall k, i \leq k < j, \sigma, k \models p$$

Abréviations LTL :

$F \Phi \equiv \text{True} \cup \Phi$

$G \Phi \equiv \neg F \neg \Phi$

7. CTL (Computation Tree Logic)

- Ajout des quantificateurs de chemins : **A** (tous les chemins), **E** (existe un chemin)
- Chaque opérateur temporel doit être précédé d'un **A** ou **E** : AX, EX, AF, EF, AG, EG, A[$\Phi \cup \Psi$], E[$\Phi \cup \Psi$]
- LTL raisonne sur une **seule exécution**, CTL sur un **arbre d'exécutions**.

8. Propriétés de Sûreté et Vivacité

♦ Tableau complet : Sûreté (Safety) et Vivacité (Liveness)

Type	Sous-type / Concept	Formule LTL	Formule CTL	Exemple	Explication
Sûreté (Safety)	Interdiction / Sûreté pure	$G \neg P$	$AG \neg P$	Un système ne doit jamais dépasser 100°C : $\neg \text{Overheat}$	"Quelque chose de mauvais ne se produit jamais"
	Invariant	$G P$	$AG P$	Le voyant "Ready" est toujours allumé	P doit être vrai tout le temps
	Sûreté conditionnelle	$G (Q \Rightarrow \neg P)$	$AG (Q \Rightarrow \neg P)$	Si le moteur tourne (Q), alors la valve ne peut pas être ouverte ($\neg P$)	"Si condition Q, alors événement dangereux P ne peut pas arriver"
	Sûreté d'ordre (strict)	$G (Q \Rightarrow X P)$	$AG (Q \Rightarrow AX P)$	Après ouverture d'une porte (Q), la lumière doit s'allumer au prochain état (P)	"Si Q arrive, l'état suivant doit vérifier P"

	Sûreté intervalle	$G (P \Rightarrow F Q)$	$AG (P \Rightarrow AF Q)$	Après un signal P, Q doit arriver avant que le système change d'état critique	"P doit être suivi de Q dans un intervalle"
Vivacité (Liveness)	Terminaison / Atteignabilité	$F P$	$EF P$	La tâche se termine : Fin = P	"Quelque chose de bon finit par arriver"
	Garantie de réponse (réactivité)	$G (P \Rightarrow F Q)$	$AG (P \Rightarrow AF Q)$	Si une requête P, une réponse Q arrivera forcément	"Chaque demande est suivie d'une réponse"
	Vivacité faible (répétition infinie)	$GF P$	$AG (AF P)$	P se produit infiniment souvent, par ex. heartbeat	"Le système revient toujours à P"
	Maintenance / Convergence	$FG P$	$AF (AG P)$	Après un certain temps, P devient toujours vrai, par ex. système stabilisé	"Au bout d'un moment, le système reste stable"
	Équité faible	$G F P$	$AG AF P$	Chaque processus P aura au moins une fois accès à la ressource	"Tous les processus finissent par être servis"
	Équité forte	$GF P$	$AG AF P$	Chaque processus P sera servi infiniment souvent	"Chaque processus sera servi régulièrement pour toujours"
Ordre / Dépendance (Until)	Livraison garantie	$P U Q$	$A [P U Q]$	L'alarme P reste active jusqu'à l'appui sur Stop Q	"P reste vrai jusqu'à ce que Q se produise, Q finira par arriver"

	Livraison possible	–	E [P U Q]	Il existe un chemin où P reste vrai jusqu'à ce que Q arrive	"P reste vrai jusqu'à Q sur au moins un chemin"
--	--------------------	---	-----------	---	---

♦ **Notes et explications complémentaires**

1. **Safety vs Liveness :**

- **Sûreté** → "rien de mauvais ne se produit" → G / AG
- **Vivacité** → "quelque chose de bon finit par arriver" → F / EF

2. **Équité (fairness) :**

- **Faible** → chaque événement finit par arriver **au moins une fois**
- **Forte** → chaque événement revient **infiniment souvent**

3. **Until / Ordre :**

- Important pour séquences obligatoires d'événements.
- LTL est linéaire → le chemin est unique, CTL est arborescent → quantificateurs A / E définissent tous les chemins ou au moins un chemin.

MAGUEMOUN SAMY

Chapitre 4 : Model Checking

Le model checking est une méthode de vérification formelle qui consiste à confronter un modèle du système à une propriété donnée, afin de vérifier automatiquement si le modèle satisfait cette propriété (notation : $A \models \Phi$ ou $A \models \Phi$). Contrairement aux tests classiques, le model checking est **exhaustif** : il prend en compte tous les comportements possibles du système.

- Pour une propriété existentielle (ex. **EF P**), il suffit de trouver **au moins un chemin** où P sera vrai.
- Pour une propriété universelle (ex. **AG P**), il faut démontrer que **tous les chemins possibles** à partir de l'état courant satisfont P en permanence.

Comme il est impossible de tester les chemins un par un (ils peuvent être très nombreux, voire infinis), le model checking repose sur des **algorithmes automatiques** capables de vérifier ces propriétés **en temps fini**.

Model Checking de CTL

En CTL, toutes les formules s'appliquent **aux états** du système.
Elles sont soit :

- des propositions atomiques (ou combinaisons booléennes),
- des formules temporelles de type **AG, EG, AF, EF, AX, EX**.

Principe du model checking CTL :

- **Données** :
 - un système AAA (représenté par une structure de Kripke),
 - une propriété PPP exprimée en CTL.
- **But** : déterminer l'ensemble des états du système AAA qui satisfont la propriété PPP.

Méthode :

On utilise l'**algorithme de marquage** du model checking CTL.

Cet algorithme travaille directement sur le modèle et consiste à **marquer progressivement les états** qui satisfont les différentes sous-formules de PPP, jusqu'à identifier les états satisfaisant la propriété complète.

Algorithme de marquage (les algos sont dans le cours aussi)

Procédure marquage (phi)

% utilise un tableau indexé par les états q de l'automate A et les sous-formules psi de phi
% dont les éléments sont notés q.psi
% on notera app(p, q) si la propriété p appartient à l'état q

Cas $\phi = p$ Alors Faire pour tout q dans Q
 Faire Si **app** (p, q) Alors Faire $q.\phi :=$
true
 Sinon Faire $q.\phi :=$ **false**

Cas $\phi = \text{not } \psi$ Alors Faire **marquage** (ψ);
 Pour tout q dans Q Faire $q.\phi :=$ **non** $q.\psi$

Cas $\phi = \phi_1 \text{ and } \phi_2$ Alors Faire **marquage** (ϕ_1);
marquage (ϕ_2);
 Pour tout q dans Q Faire $q.\phi := q.\phi_1$ **et** $q.\phi_2$

Cas $\phi = \phi_1 \text{ or } \phi_2$ Alors Faire **marquage** (ϕ_1);
marquage (ϕ_2);
 Pour tout q dans Q Faire $q.\phi := q.\phi_1$ **ou** $q.\phi_2$

Cas $\phi = E \phi_1 \text{ } \phi_2$ Alors Faire
 % on utilisera une liste L des états pour lesquels ϕ est vraie,
 % construite en parcourant à l'envers sans boucler les transitions
 de
 % l'automate.
marquage (ϕ_1); **marquage** (ϕ_2); $L :=$ vide;
 Pour tout q dans Q Faire $q.\text{marque} :=$ **false**; $q.\phi :=$ **false**.
 Pour tout q dans Q Faire Si $q.\phi_2$ Alors Faire $L := L \cup \{q\}$
 Tant que $L \neq$ vide Faire $q := \text{choix}(L)$; $L := L \setminus \{q\}$; $q.\phi :=$ **true**;
 Pour toute transition (q', q)
 Faire Si **non** $q'.$ **marque** Alors
 Faire $q'.$ **marque** $:=$ **true**;
 Si $q'.$ ϕ_1 alors
 Faire $L := L \cup \{q'\}$

Cas $\phi = E X \psi$ Alors Faire **marquage** (ψ);
 Pour tout q dans Q
 Faire Si il existe une transition (q, q') tq
 $q'.$ **psi** $=$ **true**
 Alors Faire $q.\phi :=$ **true**
 Sinon Faire $q.\phi :=$ **false**

Cas $\phi = A X \psi$ Alors Faire **marquage** (ψ);
 Pour tout q dans Q
 Faire Si il existe une transition (q, q') tq
 $q'.$ **psi** $=$ **false**
 Alors Faire $q.\phi :=$ **false**
 Sinon Faire $q.\phi :=$ **true**

Cas $\phi = A \phi_1 \text{ } \phi_2$ Alors Faire
 % on notera par $\text{degré}(q)$ le nombre de successeurs de l'état q
 dans
 % l'automate. On utilisera un tableau $q.\text{nb}$ pour parcourir tous les
 % successeurs d'un état.
marquage (ϕ_1); **marquage** (ϕ_2); $L :=$ vide;
 Pour tout q dans Q Faire $q.\text{nb} := \text{degré}(q)$; $q.\phi :=$ **false**.
 Pour tout q dans Q Faire Si $q.\phi_2$ Alors Faire $L := L \cup \{q\}$
 Tant que $L \neq$ vide Faire $q := \text{choix}(L)$; $L := L \setminus \{q\}$; $q.\phi :=$ **true**;
 Pour toute transition (q', q)
 Faire $q'.$ **nb** $:= q'.$ **nb** - 1
 Si $q'.$ **nb** $= 0$ et **Non** $q'.$ **phi** et $q'.$ ϕ_1 Alors
 Faire $L := L \cup \{q'\}$

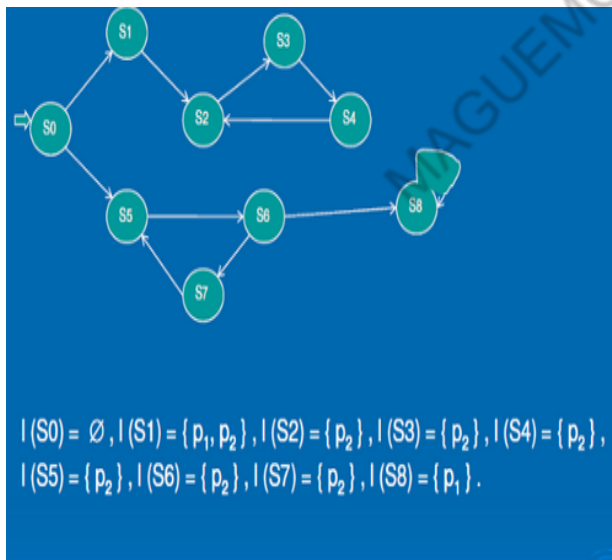
Les transformations :

$A p = \neg E \neg p$ et $F p = \text{true } U p$ et $G p = \neg F \neg p$

- $E P \equiv \neg A \neg P$

Exemple d'application de l'algorithme de marquage

- **Formule à vérifier :**
 $\phi = AF(p_1 \text{ ou } (EX \neg p_2))$
- **Étape 1 : Transformer AF en A U**
 Donc $\phi \Leftrightarrow A(T \cup (p_1 \text{ ou } (EX \neg p_2)))$
- **Étape 2 : Extraire les sous-formules dans l'ordre de marquage**
 1. p_1
 2. p_2
 3. T (toujours vrai)
 4. $\neg p_2$
 5. $EX \neg p_2$
 6. $p_1 \text{ ou } (EX \neg p_2)$
 7. $A(T \cup (p_1 \text{ ou } (EX \neg p_2)))$
- **But :** Vérifier la propriété ϕ sur une structure de Kripke donnée en marquant progressivement les états qui satisfont chaque sous-formule, en commençant par les propositions atomiques et en remontant vers la formule complète.



• Marquage:

	p_1	p_2	T	$\neg p_2$	$EX \neg p_2$	$p_1 \vee EX \neg p_2$	ϕ
S0	F	F	V	V	F	F	V
S1	V	V	V	F	F	V	V
S2	F	V	V	F	F	F	F
S3	F	V	V	F	F	F	F
S4	F	V	V	F	F	F	F
S5	F	V	V	F	F	F	V
S6	F	V	V	F	V	V	V
S7	F	V	V	F	F	F	V
S8	V	F	V	V	V	V	V

A l'état initial S0, ϕ est vraie, donc le modèle satisfait ϕ .

$I(S0) = \emptyset, I(S1) = \{p_1, p_2\}, I(S2) = \{p_2\}, I(S3) = \{p_2\}, I(S4) = \{p_2\},$
 $I(S5) = \{p_2\}, I(S6) = \{p_2\}, I(S7) = \{p_2\}, I(S8) = \{p_1\}.$

Sûreté, Vivacité et Équité

- Une propriété de **sûreté** exprime que « quelque chose de mauvais ne doit pas arriver ».
- Une propriété de **vivacité** exprime que « quelque chose de bon doit inévitablement arriver ».
- Une propriété d'**équité** exprime une règle de justice entre plusieurs utilisateurs d'une même ressource ou d'un même service.